

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 4

Total Marks:40

Annexure A

ARTICLE REVIEW

Code of Conduct:

*I **Shaik Fahed (192565026)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SHAIK FAHED

ARTICLE REVIEW

INSTRUCTIONS

Select any ONE peer-reviewed research article (published after 2019) from IEEE Xplore, Springer, Elsevier, or ACM Digital Library related to any of the following topics: *Inductive Learning, Decision Trees, Statistical Learning Methods (Naïve Bayes / Logistic Regression / SVM), or Reinforcement Learning applications*.

Submit: (i) Article citation details (ii) PDF or valid DOI link (iii) Written review as per the tasks below.

Task 1: Article Summary

(a) Provide the full citation of the article (Author(s), Title, Journal/Conference, Year, DOI). State the domain/application area and the specific ML problem addressed.

(b) Summarise the key objective of the article in your own words (150–200 words). Clearly state the research gap the authors identified and how they proposed to fill it.

Task 2: Methodology Analysis

(a) Explain the ML technique(s) used in the article (e.g., Decision Tree variant, RL algorithm, Statistical model). Describe the dataset used—size, features, source, and how it was prepared.

(b) Map the technique to CO 4 topics (Inductive Learning / Decision Trees / Statistical Learning / RL). Identify one methodological strength and one limitation in the authors' approach.

Task 3: Results & Critical Evaluation

(a) Report the key results (accuracy, F1-Score, AUC-ROC, reward convergence, etc.) as presented by the authors. Create a comparative table if multiple models are benchmarked.

(b) Critically evaluate the results: Are the reported metrics realistic? Are there potential biases (dataset, evaluation protocol)? What additional experiments would strengthen the findings?

(c) Discuss real-world applicability: Where can this technique be deployed? What are the challenges in scaling the solution to industry (data availability, compute, ethical issues)?

Task 4: Reflection & Learning Outcome

(a) State three key insights you gained from reading the article that deepen your understanding of CO 4 topics. Connect each insight to a specific concept from the course syllabus.

(b) If you were to extend this research, propose one novel experiment or enhancement (different dataset / improved algorithm / new application domain). Justify its feasibility and expected impact.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Article	20%	Demonstrates clear and in-depth understanding of the article's purpose, concepts, and arguments	Shows good understanding with minor gaps	Partial understanding; misses key ideas	Lacks understanding of the article
Summary	20%	Provides concise and accurate summary highlighting all key points	Covers most key points with minor omissions	Incomplete or partially accurate summary	Poor or incorrect summary
Critical Analysis	25%	Provides insightful critique with strong reasoning and evaluation	Provides reasonable analysis with some justification	Superficial analysis; limited evaluation	No meaningful analysis
Use of Evidence	15%	Effectively uses examples, data, or references to support review	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Organization	20%	Well-structured, clear, and coherent writing with proper language and flow	Generally clear with minor issues	Poor organization; lacks clarity	Disorganized and difficult to understand

INTERACTIVE REINFORCEMENT LEARNING FOR FEATURE SELECTION USING DECISION TREE FEEDBACK

College Assignment — Extended Edition

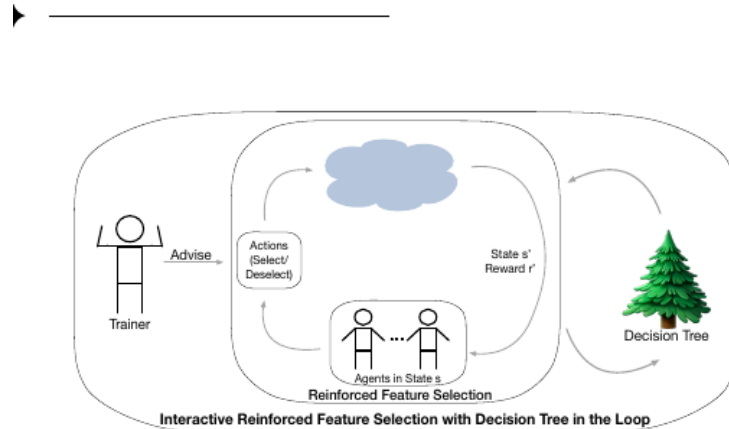


Fig. 1: Interactive reinforced feature selection via interaction with external trainers and a downstream task (decision tree).

Abstract

Feature selection is one of the most consequential preprocessing steps in machine learning: choosing a compact, informative subset of attributes from a much larger pool directly affects model accuracy, training time, and interpretability. This assignment presents and expands upon an interactive reinforcement-learning approach in which individual agents — one per candidate feature — decide whether their feature should be kept or dropped, while receiving guidance from K-Best and Decision Tree trainers along the way. Decision-tree information is used twice over: once to enrich how the current feature subset is represented as a state, and once to shape the reward each agent receives. Across both uses, the goal is the same — combine the exploratory power of reinforcement learning with the structural insight of a downstream classifier so that good feature subsets are found faster and more reliably than either classical filter methods or unguided reinforced search can manage alone. This extended write-up walks through the motivation, the framework, every trainer and reward variant, the datasets used for evaluation, and the reported results, illustrated throughout with the original figures from the source study.

1. Introduction

Feature selection reduces the number of attributes a machine-learning model has to consume while trying to preserve — or even improve — predictive performance. Removing irrelevant or redundant columns from a dataset speeds up training, reduces overfitting, and often makes the resulting model easier to interpret. Classic approaches such as K-Best, mRMR (minimum-Redundancy-Maximum-Relevance), LASSO, and recursive feature elimination remain popular precisely because they are fast and simple to apply. However, all of them share a common weakness: they evaluate each feature more or less independently, or rely on a fixed scoring rule, so they can miss combinations of features whose joint value is greater than the sum of their individual scores.

Reinforcement learning (RL) offers a fundamentally different way to search the feature space. Instead of ranking features once and cutting a threshold, an RL-based selector treats feature selection as a sequential decision-making problem: at each step, one or more agents choose to include or exclude features, observe the effect on downstream accuracy, and adjust their policy through trial and reward. This turns feature selection into an exploration problem, which is attractive because it can, in principle, discover subsets that simple ranking schemes would never propose. The catch is efficiency — naive RL exploration can take a very large number of steps to converge, especially as the number of candidate features grows into the hundreds or thousands.

This assignment is built around a method that tries to keep the exploratory strength of reinforcement learning while curbing its inefficiency. It does so by letting each feature-selecting agent ask for help. Two classical feature-selection techniques — K-Best and Decision Tree — are recast as "trainers" that can step in and correct an agent's tentative decision before it is finalized. On top of that, the paper this assignment is based on goes one step further: it feeds the internal structure of a decision tree, trained on the currently selected features, back into the reinforcement-learning loop itself, both to describe the state more richly and to divide the reward more fairly among agents.

2. Problem Statement

Formally, given a set of N candidate features $F = \{f_1, f_2, \dots, f_N\}$, the goal is to find a subset $F' \subseteq F$ that gives the best possible performance on a downstream predictive task, without exhaustively searching the 2^N possible subsets. The central tension the assignment explores is a dilemma observed by the original authors in their preliminary work: classical feature-selection methods (mRMR, K-Best, and similar) are computationally cheap but tend to settle for a mediocre subset because they do not model interactions between features; reinforced feature selection can, given enough exploration, discover a much better subset, but "enough exploration" can mean an impractical number of steps, particularly as feature count increases.

The proposed approach addresses this dilemma from two directions simultaneously. First, it allows reinforcement-learning agents to interact with external trainers, so that agents who are uncertain about a decision can be nudged toward a better one instead of relying purely on random exploration. Second, it closes the loop between the reinforcement-learning process and the downstream decision tree: rather than treating the decision tree purely as an evaluator that reports a final accuracy number, the framework mines the tree's internal structure — which

features it splits on, and how important each feature turned out to be — and reuses that information to make the agents' state representation and reward signal more informative.

3. Proposed Interactive Reinforcement-Learning Framework

Each input feature is associated with its own agent, and every agent carries its own policy network (a small Deep Q-Network) that decides, at each exploration step, whether its corresponding feature should be selected or deselected. Because every agent only has to make a binary decision about one feature, the action space per agent stays small and manageable, in contrast to a single-agent design where the action space would need to cover all 2^N combinations at once.

Agents that are hesitant about their decision — meaning they were selecting the feature on the previous step but are now leaning toward dropping it — can receive advice from one or more trainers. The overall selected feature subset that results from all agents' actions is evaluated by a downstream learning task (a decision tree classifier in this framework), and the resulting predictive performance, together with structural feedback from the tree, is converted into a reward signal that is fed back to every participating agent so their policies can keep improving.

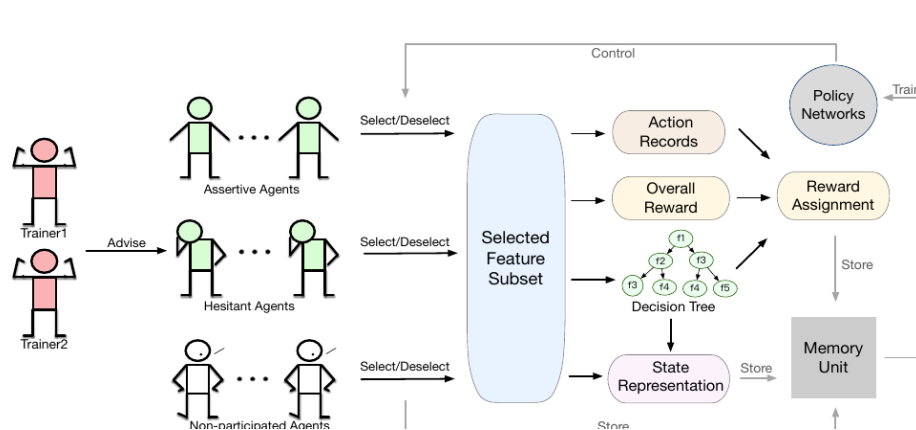


Fig. 2: Framework overview. Agents select or deselect their corresponding features based on their policy networks and trainers' advice. Action, reward, and state are stored in a memory unit used to train the policy networks. The decision tree's structure feeds back into both state representation and reward measurement.

Reading Figure 2 left to right: two trainers advise the assertive and hesitant agents (non-participated agents act without advice); every agent's select/deselect decision updates the selected feature subset; that subset feeds three downstream processes — action records, an overall reward signal, and state representation, each partly informed by a decision tree trained on the current subset — and the resulting reward assignment and state are written to a shared memory unit that trains the policy networks for the next round.

4. Role of Decision Tree Feedback

A decision tree is retrained at every exploration step using only the currently selected features. Because a decision tree splits on the feature that most reduces impurity at each node, its resulting feature-importance scores summarize, in a single number per feature, how useful that feature actually is for prediction given the company it is currently keeping — as opposed to its usefulness in isolation, which is all a filter method like K-Best can measure.

This structural information is fed back to the reinforcement-learning agents in two separate ways. First, it augments the state representation: instead of representing the selected subset purely as an undirected graph of pairwise feature correlations, the framework overlays a directed tree of decision splits, so agents can "see" not just which features are currently selected but how those features relate to one another inside the classifier that will ultimately judge them. Second, it feeds directly into the reward calculation, so that an agent responsible for selecting a feature the tree considers important receives a larger reward than an agent whose feature turned out to be nearly irrelevant to the tree's splits. Both uses are explained in more depth in Sections 8 and the state-representation discussion below.

5. K-Best and Decision Tree Trainers

The framework's advice mechanism starts by classifying agents at every step into three groups. Participated agents are those whose feature was selected on the previous step; everyone else is non-participated and simply acts on its own policy without receiving advice. Among the participated agents, assertive agents are the ones whose policy network still wants to keep the feature selected, while hesitant agents are the ones whose policy network has just decided to drop a feature it was keeping a moment ago. Only hesitant agents are eligible to receive advice — the idea being that an agent flip-flopping away from a feature is the one most likely to be making a mistake that a trainer can catch.

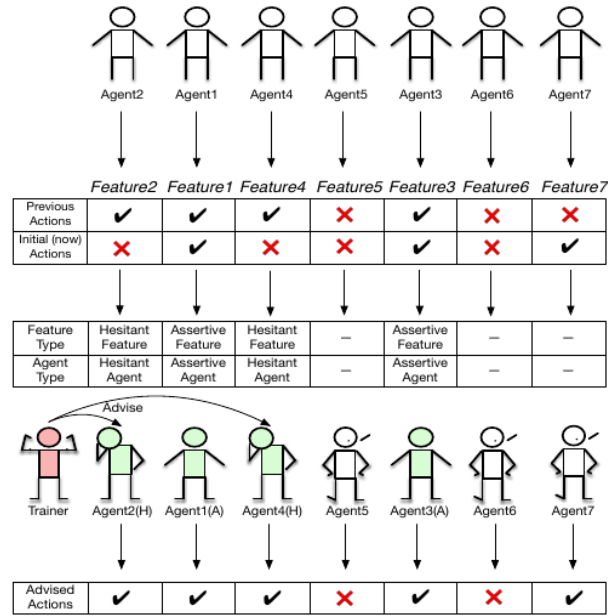


Fig. 3: General process of a trainer guiding agents. The framework first labels agents as assertive ('A') or hesitant ('H'); the trainer then offers advice only to the hesitant agents.

The K-Best trainer ranks the participated features using a fast filter score (mutual information, in the reported experiments) and asks: if a hesitant feature is even better than half of the currently assertive features, its agent should be told to reselect it. Concretely, the trainer computes $k = \lceil m/2 + n \rceil$, where m is the number of assertive features and n is the number of hesitant features, then keeps the top k participated features by K-Best score. Any hesitant feature that lands inside that top- k set gets overridden back to "select."

The Decision Tree trainer follows the same two-phase structure — identify hesitant/assertive agents, then advise — but swaps the evaluation criterion. It trains a decision tree on the participated features, reads off each feature's importance score, and computes the median importance among the assertive features as a threshold g . Any hesitant feature whose importance exceeds g is considered comparatively important and is advised back to "select." Because a decision tree's importance scores are sensitive to interactions between features (a feature can matter a great deal only in combination with another), this trainer tends to catch cases that a purely univariate method like K-Best would miss.

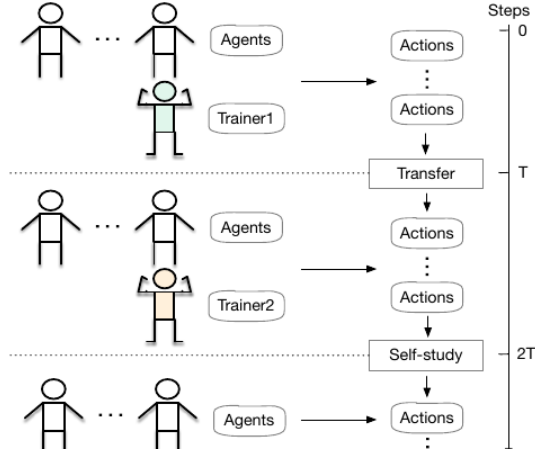


Fig. 4: General process of the Hybrid Teaching strategy — one trainer advises first, a second trainer takes over later, and finally the agents explore and learn on their own.

6. Hybrid Teaching Strategy

Rather than committing to a single trainer for the whole training run, the framework can combine the knowledge of both trainers through a Hybrid Teaching strategy, loosely modeled on how human students progress through different teachers at different stages of schooling. From step 0 to step T the KBest trainer leads; from step T to step $2T$ the Decision Tree trainer takes over; from step $2T$ onward the trainers step back entirely and the agents explore and refine their policies by themselves, relying only on the experience already accumulated in memory.

The motivation for alternating trainers, rather than picking whichever one performs best and sticking with it, is diversity. A single trainer tends to give very similar advice whenever it sees similar participated features, which can trap the exploration process in a narrow region of the search space. Introducing a second trainer with a different evaluation criterion breaks that pattern and reduces the chance that one trainer's blind spot becomes the whole system's blind spot. At the same time, handing control back to the agents in the final phase prevents them from becoming permanently dependent on external advice, preserving their ability to keep improving unsupervised once useful habits have been learned.

7. State Representation and GCN

How the environment's current state is represented matters just as much as how rewards are computed, because the policy network can only make good decisions based on what it is shown. The framework's baseline approach treats the selected features as a fully-connected graph $G(V)$, where every feature is a node and every edge is weighted by the Pearson correlation coefficient between the two features it connects. A Graph Convolutional Network (GCN) is then used to propagate information across this graph and produce an updated representation for each node.

ig. 4: General process of Hybrid Teaching strategy. One trainer gives advice firstly; then, another trainer gives advice. Finally, agents explore and learn by themselves.

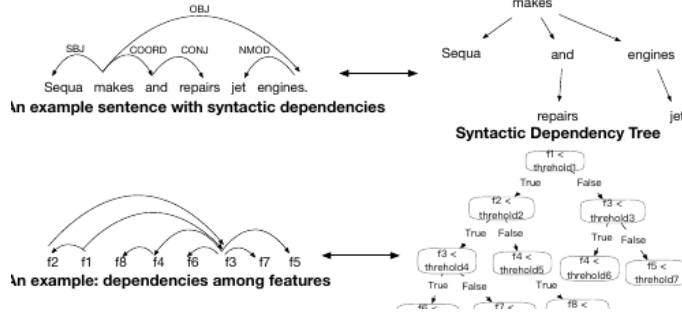


Fig. 5: Analogy drawn from NLP — just as a syntactic dependency tree captures grammatical relationships in a sentence, a decision tree captures dependency relationships among selected features.

The extension explored in this assignment borrows an idea from syntactic graph convolutional networks in natural-language processing, where a dependency tree over a sentence's words is used to sharpen a language model's representation of that sentence. Analogously, the decision tree trained on the current feature subset is treated as a directed tree of "decision dependencies" among features, and its edges are overlaid on top of the correlation graph. The result, $G(V)'$, keeps the original undirected correlations but adds directed edges wherever the decision tree actually split from one feature to another, giving the GCN a second, complementary source of structure to learn from.

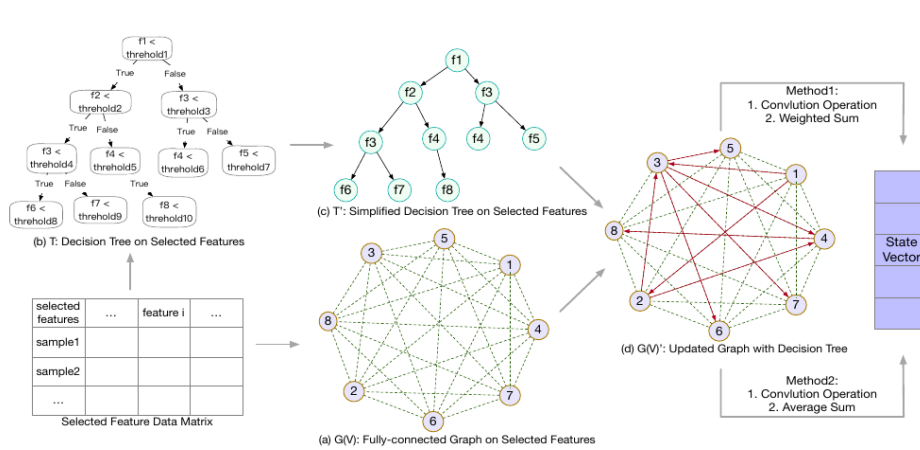


Fig. 6: General process of state representation with decision tree structure feedback — from the fully-connected correlation graph (a) and the raw decision tree (b), through a simplified tree (c), to the combined graph $G(V)'$ (d) that feeds the state vector.

Node representations are updated with an enhanced convolution operation that blends two sources: the ordinary correlation-graph neighborhood, and the new decision-tree neighborhood, weighted by a tuning parameter λ . Once every node has an updated representation, two alternative readout methods are proposed to collapse the whole graph into a single state vector: Method 1 takes a weighted sum of node representations, weighted by each feature's importance in the decision tree; Method 2 simply averages all node representations uniformly. These correspond to the SRDT1 and SRDT2 variants evaluated later in the results section.

8. Personalized Reward

Traditional multi-agent reinforced feature selection tends to split one overall reward equally among all agents that selected a feature, regardless of how much any individual feature actually contributed to that reward. The framework instead uses personalized reward schemes, so that a feature that is repeatedly selected or is strongly associated with prediction receives a larger share of the credit than a feature that happened to be along for the ride.

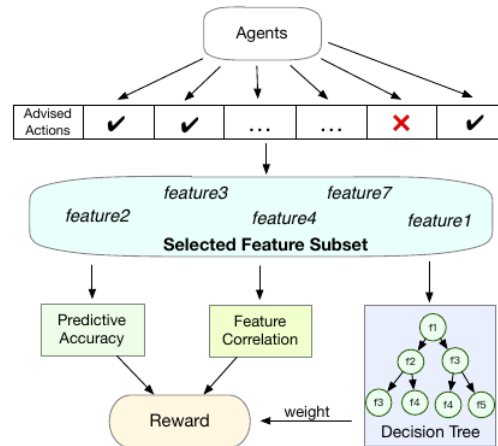


Fig. 7: Measuring reward using feature importance from the decision tree — predictive accuracy and feature correlation are combined and then weighted per agent by tree importance.

The first personalized scheme weights each agent's reward by the feature importance reported by a decision tree trained on the selected subset at that step. Two properties of the subset are folded into the base reward before this weighting is applied: predictive accuracy (higher is better) and feature correlation (lower average correlation is better, since a good subset should not be full of redundant, highly-correlated features). An agent that selected a feature the tree found important receives a proportionally larger reward than one whose feature barely mattered to the tree's splits; agents that deselected their feature receive zero reward for that step, since they contributed nothing to the subset being evaluated.

7

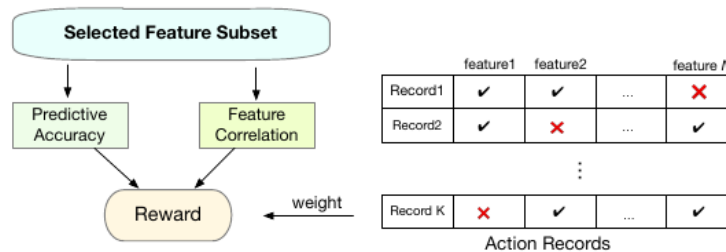


Fig. 8: General process of measuring reward with historical

Fig. 8: Measuring reward using historical action records — how consistently a feature has been selected over past steps is used to weight the reward it receives now.

The second personalized scheme takes a complementary view: instead of asking a decision tree how important a feature is right now, it asks how consistently the corresponding agent has chosen to select that feature across past exploration steps. A feature that shows up in the selected subset over and over again is treated as important almost by definition — persistent agent behavior is itself evidence of value — and its selected-frequency ratio is used to weight the reward. These correspond to the PRS1 (decision-tree-weighted) and PRS2 (history-weighted) variants referenced in the results below, and the framework also evaluates SRDT+PRS, which combines the best state-representation method with the best reward scheme for a further boost.

9. Working Procedure

1. Input the dataset and identify its available features, one agent per feature.
2. Create an agent for each feature and allow every agent's policy network to select or deselect its corresponding feature.
3. Separate the participated agents into confident (assertive) and hesitant groups based on whether their decision changed from the previous step.
4. Use the K-Best trainer and/or the Decision Tree trainer — alone or through the Hybrid Teaching strategy — to provide advice to hesitant agents.
5. Evaluate the resulting selected feature subset using the downstream decision tree classifier.
6. Update the feature-subset representation using the feature-feature correlation graph, the decision tree structure, and the GCN.
7. Calculate personalized rewards for every agent and store the (state, action, reward, next state) experience tuple in the memory unit.
8. Train each agent's policy network on mini-batches sampled from the memory unit, using the Bellman equation to target long-term reward.
9. Repeat the exploration loop for the configured number of steps until an improved, stable feature subset is obtained.

This nine-step loop is what runs, unchanged, for every variant discussed in this assignment — the differences between MARLFS, KBT, DTT, HT, SRDT1/2, PRS1/2, and SRDT+PRS all live inside steps 4, 6, and 7. Keeping the outer loop fixed is what makes the reported comparisons fair: every variant sees the same number of exploration steps, the same downstream classifier, and the same datasets.

10. Experimental Evaluation

The study evaluates the method on eight publicly available, real-world datasets that between them span two orders of magnitude in feature count (16 to 1,024) and roughly an order of magnitude in sample count (4,601 to 34,465). This range matters because it lets the experiments show whether the interactive framework's advantages hold up as the feature space grows from something a filter method could brute-force (16 features) to something genuinely hard to explore exhaustively (1,024 features).

Dataset	Features	Samples	Task Type
PRD	16	10,992	Digit classification (10 classes)

Forest Cover	54	15,120	Wilderness cover-type classification
Spam	57	4,601	Binary spam / not-spam classification
ICB	86	5,000	Insurance product-usage prediction
Nomao	120	34,465	Binary place-matching classification
Musk	168	6,598	Musk / non-musk molecule classification
ESR	178	11,500	Epileptic seizure recognition (5 classes)
QSAR	1024	8,992	Oral toxicity (toxic / non-toxic)

Each dataset targets a different real-world classification problem: PRD is handwritten-digit recognition; Forest Cover predicts wilderness cover type from cartographic variables; Spam is email spam detection; ICB predicts insurance product usage from demographic and policy data; Nomao is a place-deduplication task; Musk distinguishes musk from non-musk molecules; ESR flags epileptic seizure activity in EEG-derived features; and QSAR predicts oral toxicity of chemical compounds, the largest feature space of the eight. For every dataset, 80% of samples were used for training and 20% for testing, with the downstream decision tree left at scikit-learn's default hyperparameters so that comparisons across feature-selection methods are not confounded by classifier tuning.

11. Results

The headline result is that the full Interactive Reinforced Feature Selection (IRFS) method achieves the best overall Best-Accuracy performance on all eight evaluated datasets, beating five baselines: three classical filter/embedded methods (K-Best, mRMR, LASSO), one wrapper method (Decision Tree Recursive Feature Elimination, DT-RFE), and the un-guided reinforced baseline MARLFS.

9

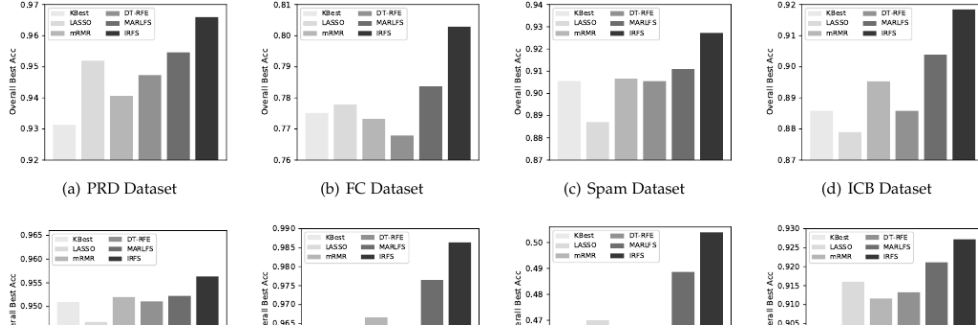


Fig. 9: Overall Best Accuracy of different feature-selection algorithms across all eight datasets. IRFS (rightmost bar, darkest) leads on every dataset.

Two patterns stand out in Figure 9. First, MARLFS — the plain reinforced baseline with no trainer at all — already beats the four traditional methods on most datasets, which supports the paper's starting premise that globally optimizing over the feature subspace, rather than scoring features independently, is worth the extra computation. Second, IRFS consistently improves further on top of MARLFS, which is the more interesting claim: it shows that trainer guidance and decision-tree feedback are not just a shortcut to the same ceiling MARLFS would eventually reach, but actually raise that ceiling.

10

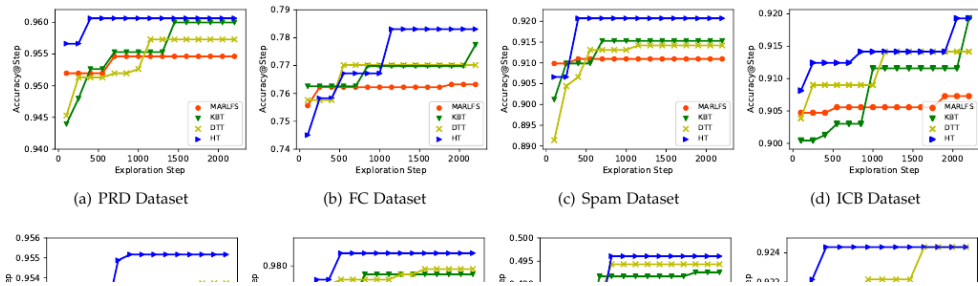


Fig. 10: Exploration efficiency comparison — accuracy achieved after a given number of exploration steps, for MARLFS, KBT, DTT, and HT.

Figure 10 speaks to efficiency rather than final accuracy: both KBT and DTT reach higher accuracy than MARLFS within relatively few steps (around 2,000), showing that trainer guidance genuinely speeds up the search rather than only improving its eventual endpoint. IRFS with Hybrid Teaching (HT) tends to find good subsets in the shortest time of all four variants on most datasets, consistent with the interpretation that blending two trainers lets the system draw on a broader base of knowledge than either trainer offers alone.

11.1 Study of State Representation and Reward Variants

Beyond the headline comparison, the paper isolates the individual contributions of the two decision-tree-feedback mechanisms — state representation and reward — through five additional variants, summarized below.

Variant	What it adds on top of MARLFS
MARLFS	Baseline multi-agent reinforced feature selection; no trainer, no decision-tree feedback.
IRFS + KBT	Adds a KBest-based trainer that advises hesitant agents using mutual-information ranking.
IRFS + DTT	Adds a Decision-Tree-based trainer that advises hesitant agents using tree feature importance.
IRFS + HT	Hybrid Teaching: KBT and DTT each take the lead for part of training, then agents self-study.
IRFS + SRDT1 / SRDT2	Replaces the plain feature-feature graph state with a GCN state that also encodes the decision tree (weighted-sum vs. average-sum readout).
IRFS + PRS1 / PRS2	Replaces equal reward-sharing with tree-importance-weighted reward (PRS1) or action-history-weighted reward (PRS2).
IRFS + SRDT+PRS	Combines the best state-representation method with the best reward scheme for the largest overall gain.

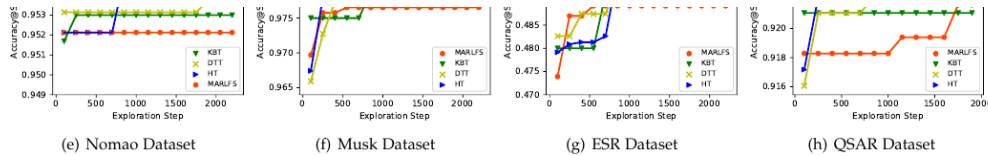


Fig. 10: Exploration efficiency comparison of IRFS methods.

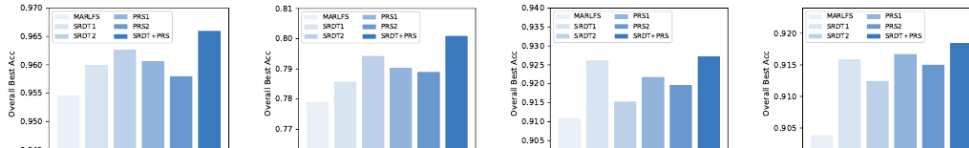


Fig. 11: Best Accuracy comparison of the variant methods against the MARLFS baseline, across all eight datasets.

Every one of the five variants improves on the MARLFS baseline, on every dataset, which confirms that both proposed mechanisms — richer state representation and personalized reward — independently contribute useful signal. The combination SRDT+PRS is the strongest variant in most cases, suggesting the two mechanisms are complementary rather than redundant: knowing more about the current subset (state) and rewarding agents more precisely for their actual contribution (reward) attack different parts of the same exploration problem.

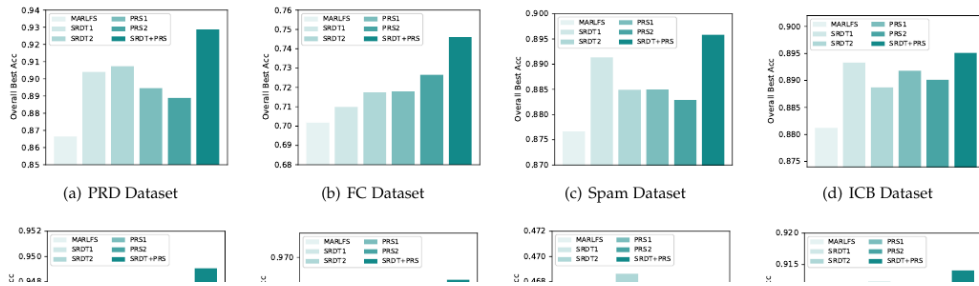


Fig. 12: Average Accuracy (Ave Acc) comparison of the variant methods, showing consistency of performance over time rather than just the best step reached.

Best Acc measures the peak the exploration process reaches; Ave Acc measures how good the subsets are on average across the whole run, which is arguably a better proxy for how reliable the search is rather than how lucky it eventually gets. Figure 12 shows the same ordering holds for Ave Acc as for Best Acc — all proposed variants beat MARLFS, and SRDT+PRS again leads or ties for the lead on most datasets — which strengthens the claim that the improvement is due to genuinely better exploration quality rather than one fortunate late-stage discovery.

11.2 Study of Action-Changed Agents

As a sanity check on why the trainers help at all, the study records, at every exploration step, how many hesitant agents actually take the trainer's advice and flip their action from deselect back to select. The distribution of this count (0 to 9 agents changing per step, in the reported experiments) is visualized as a set of pie charts per dataset.

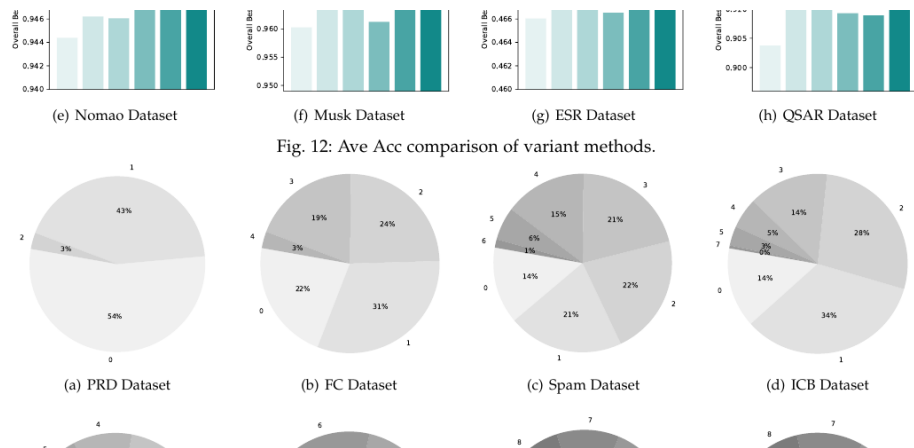


Fig. 12: Ave Acc comparison of variant methods.

Fig. 13: Percentage of exploration steps in which a given number of agents (0–9) change their action after trainer advice, per dataset.

Most steps see a non-zero number of action changes, which is the evidence that trainer advice is not just noise being ignored — it is actively steering agents toward better decisions often enough to matter. The two datasets with the most features among those charted in detail, ESR and QSAR, show noticeably more action-changed agents than the smaller datasets. The likely explanation is simply that larger feature spaces present more opportunities for an agent's initial, unguided decision to be wrong, so trainers have proportionally more corrections to make.

12. Advantages

Advantage	Why it matters
Automates feature selection	Removes the need for a human analyst to hand-pick attributes for every new dataset.
Uses external knowledge to guide exploration	Trainers inject domain-relevant signal instead of relying on random search alone.
Incorporates Decision Tree feature importance	Structural information from a downstream model sharpens both state and reward.
Provides personalized rewards	Each agent is credited according to its actual contribution, not an equal split.
Improves exploration efficiency	Good subsets are found in noticeably fewer steps than the unguided baseline.
Combines multiple trainers via Hybrid Teaching	Blending KBest and Decision Tree advice reduces the risk of one trainer's bias dominating.
Scales to comparatively large feature spaces	Tested up to 1024 features (QSAR dataset) without the search space exploding.

13. Applications

Because the framework is dataset-agnostic — it only needs a set of numeric or categorical features and a downstream classifier — it is directly relevant to a wide range of classification and data-mining problems. The eight evaluation datasets alone touch spam detection, handwritten-digit recognition, insurance-customer analysis, place/entity matching, molecular (musk) classification, epileptic-seizure detection from biosignals, and toxicity prediction for drug-discovery pipelines. More broadly, any domain where practitioners currently rely on manual feature engineering or a single filter score — genomics, fraud detection, industrial sensor monitoring, and text classification, among others — is a plausible candidate for this kind of interactive, feedback-driven feature selection.

14. Limitations

The framework is meaningfully more complex than a simple filter-based feature-selection pipeline, since it combines reinforcement-learning agents, one or more external trainers, a downstream decision tree that must be retrained repeatedly, a graph-convolutional state representation, and a personalized reward calculation. Each of these components adds its own hyperparameters (the tuning parameters λ and β among them) that need to be set sensibly, and each adds computational overhead. Repeated exploration and training, especially across thousands of steps and up to 1,024 features (as in the QSAR dataset), can require considerable computational resources — the reported experiments were run on a dedicated multi-core workstation with 128GB of RAM, which is not a trivial setup.

A further limitation implicit in the design is that all of the trainer and reward mechanisms depend on the downstream task being a decision tree, or at least a model that exposes an interpretable feature-importance signal. Extending the same ideas to downstream tasks without such structure — a deep neural network classifier, for instance — would require a different source of structural feedback, since a black-box model does not offer the same tree-structured hierarchy to feed back into the loop.

15. Conclusion

Interactive reinforcement learning offers a genuinely useful way to automate feature selection without giving up the global, combination-aware search that makes reinforcement learning attractive in the first place. By adding external trainer advice — either K-Best, Decision Tree, or the two combined through Hybrid Teaching — the framework speeds up an otherwise slow exploration process. By further closing the loop with decision-tree structural feedback, both for state representation and for reward assignment, it sharpens the signal agents learn from at every step, rather than treating the downstream classifier as a black box that only ever returns a single accuracy number.

The experimental results reported in the source study, reproduced throughout this assignment, indicate consistent improvement over traditional filter and wrapper methods (K-Best, mRMR, LASSO, DT-RFE) as well as over the un-guided reinforced baseline MARLFS, across eight real-world datasets that vary widely in feature count and sample size. The ablation-style comparison across state-representation and reward-scheme variants further shows that the two decision-tree-feedback mechanisms contribute independently and combine constructively, which is good evidence that the underlying idea — feeding a downstream model's internal structure back into the search process that built its input — generalizes beyond any single trick.

Reference

W. Fan, K. Liu, H. Liu, Y. Ge, H. Xiong, and Y. Fu, "Interactive Reinforcement Learning for Feature Selection with Decision Tree in the Loop," arXiv:2010.02506, 2020.

K. Liu, Y. Fu, P. Wang, L. Wu, R. Bo, and X. Li, "Automating Feature Subspace Exploration via Multi-Agent Reinforcement Learning," Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 207–215.

All figures reproduced in this assignment are drawn from the source arXiv paper listed above and are used here as visual references for the experimental results and framework design.